



6



12



1440 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

التعليمات الأساسية التوابع 1

د. سيرا أستور



RB Informatcs; 19/05/2025

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

إعادة:

تقسم تعليمات لغة Assembly 5 أقسام:

- Load/store
- Computational
- Floating point
- Jump and Branch
- Memory Management

وتعلمنا: Loop while, if else, if

و الشيء الذي لم نتعلمه هو switch case ولكن نستنتج من if else , if

وفي هذه المحاضرة سنتعلم عن التوابع Function وكيف سوف يتعامل الـ software و hardware مع التوابع.

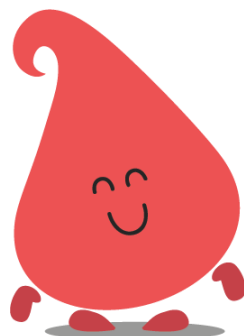
تذكرة:

التابع: هو أداة يستخدمه المبرمج لهيكله البرامج، لجعله أسهل للفهم وتحقيق الـ reusability.

توقيع التابع: هي عدد البارامترات ونوعها وترتيبها واسم التابع.

ملاحظة:

حالياً لا يتم تسميته بتوقيع التابع بل يسمى interface.



Execution of a Procedure

تنفيذ أي تابع يتم عن طريق خطوات:

1. وضع الباراميترات في مكان يستطيع التابع رؤيتها أو يستطيع الوصول إليها
2. نقل التحكم إلى Procedure (استدعاء التابع = < تغيير محتوى pc).
3. تأمين مساحة لتخزين المتغيرات التي تولد بداخله (للتابع).
4. تنفيذ المهمة المطلوبة داخل التابع.
5. وضع النتيجة في مكان يستطيع التابع الرئيسي main الوصول إليها.
6. إعادة التحكم إلى التابع الرئيسي عن طريق القفز إلى العنوان المخزن في الذاكرة كموضع العودة Return Address.

تذكرة:

- السجلات التي عادةً تستخدم لتمرير القيم \$a0-\$a3 مثل parameters وسميت هكذا اختصار لـ Argument
- السجلات التي عادةً تستخدم لإعادة القيم \$v0-\$v1
- \$ra: return address.
- القفز لعنوان jump.
- jal (jump and Link) تقوم بحفظ pc+4 في السجل \$ra كما يقوم بالقفز إلى عنوان معين في البرنامج، شكل التعليمة procedure Address jal
- تحفظ تعليمة jal pc+4 في السجل \$ra للارتداد بالتعليمة التالية.
- تستخدم تعليمة jr للقفز غير المشروط jr \$ra، كما تستعمل في الـ case statement.

خطوات تخزين القيم الذي يحتاجها التابع

- كل تابع له مساحة تخزين خاصة فيه داخل الذاكرة وتدعى هذه المساحة بالنسبة للتوابع stack
- stack**: Data structure من النوع (LIFO) Last-in-First-out تحتوي على مؤشر لآخر قيمة يدعى (sp) stack pointer.

ملاحظة:

- في MIPS أحد السجلات تدعى (\$sp) نضع به عنوان آخر عنصر بـ stack.
- ولـ stack نوعين من العمليات في اللغات عالية المستوي وهي:
- push: لوضع قيمة فيه.
 - pop: لحذف أو سحب قيمة منه.

Restore values from stack

نعلم أن نطاق التابع منفصل عن نطاق main. يمكن تعريف متحولين بنفس الاسم أحدهما داخل التابع والآخر ضمن main ولا يحدث تضارب بينهما، لذلك كل ما نستخدمه بالتابع من متحولات يتوجب علينا إعادته إلى قيمته الأصلية (restore values) عند العودة للتابع main، ومن أجل ذلك يجب حفظ قيمته قبل الذهاب إلى التابع.

■ وأخيراً نعيد القيم التي يرجعها التابع التي تكون مخزنة عادة بـ \$v0-\$v1.

■ Example:

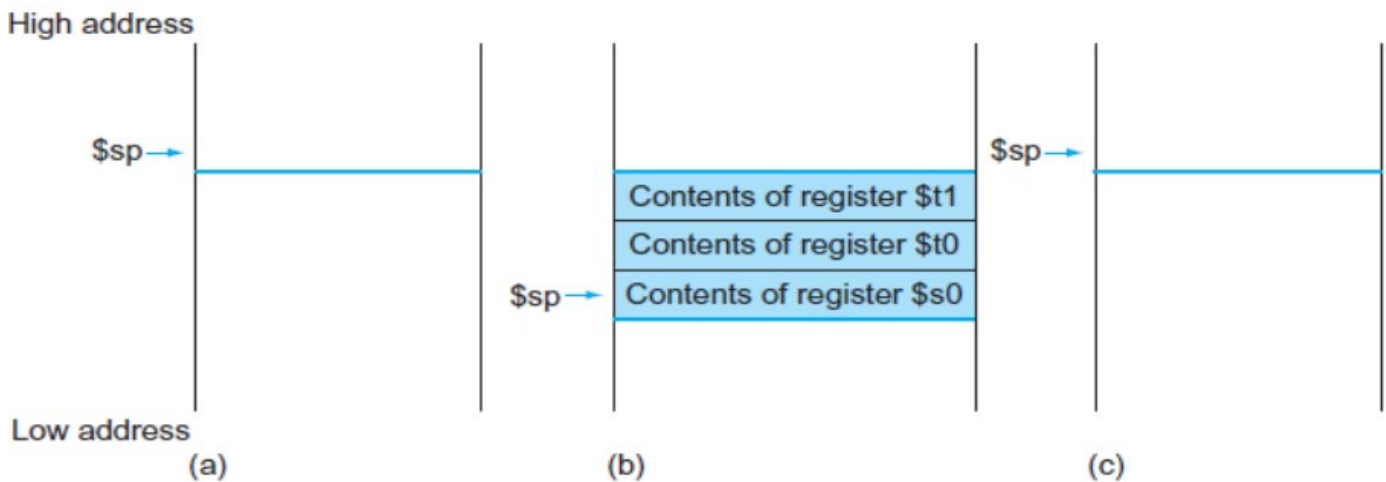
```
int leaf_example (int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

What is compiled MIPS assembly code?

يوجد لدينا 4 Parameter نخزنها بأربع سجلات وهي \$a0, \$a1, \$a2, \$a3 ثم نخزن المتحول f (النتيجة) في السجل \$s0.

■ الخطوات:

- أولاً: عملية الانتقال من main إلى التابع باستخدام الـ label الخاصة بالتابع leaf_example.
- ثانياً: نبحت عن السجلات التي سأستخدمها في التابع علماً أنه يمكن أن يكون له قيمة في main. في مثالنا: كل سجل 4 bytes بما أن لدينا 3 سجلات نحتاج 12 bytes للتخزين داخل الـ stack، تدل على رأس stack.



الحجز بالسالب ليتم حجز 3 أماكن لتغيير مكان المؤشر.

```
addi $sp, $sp, -12
sw $t1, 8($sp)
sw $t0, 4($sp)
sw $s0, 0($sp)
```

لماذا نحتاج 3 سجلات؟ لأنه في الحقيقة، يمكن للمعالج القيام بعملية رياضية واحدة على سجلين على الأكثر في المرة الواحدة و تخزين الناتج في سجل ثالث، لكي نحفظ النتائج المرحلية $i+j$, $g+h$ سنستخدم سجلان $t1$, $t0$ والناتج النهائي في $s0$ أي ثلاث سجلات. قمنا بتخزين قيمة السجلات قبل الانتقال للتابع.

■ ثالثاً: Body of the function:

```
add $t0, $a0, $a1      #t0 <= g + h
add $t1, $a2, $a3      #t1 <= i + j
sub $s0, $t0, $t1      #f <= g + h
```

■ رابعاً: Save return Address:

حفظ النتيجة النهائية بمكان يستطيع main الوصول إليها لذلك نقوم بشحن السجل $v0$ بقيمة $s0$.

```
add $v0, $s0, $zero
```

■ خامساً:

استعادة القيم الثلاث للسجلات التي حفظناها عن طريق إخراجها من stack (الذاكرة) عن طريق هذه التعليمات:

```
lw $s0, 0($sp)
lw $t0, 4($sp)
lw $t1, 8($sp)
addi $sp, $sp, 12      #نعيد المؤشر إلى مكانه
```

■ ملاحظة:

يتم التحميل من نفس مكان التخزين في stack الذاكرة.

■ سادساً:

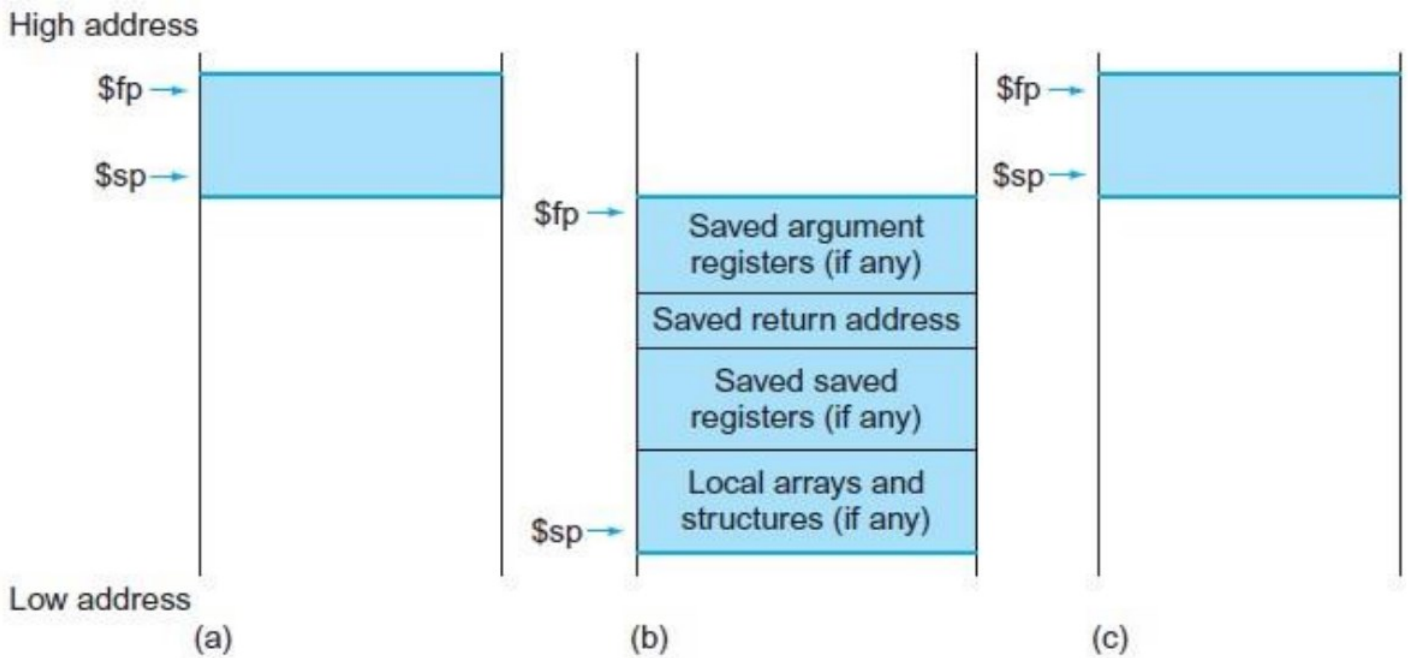
عودة التحكم إلى main عن طريق تعليمة jr.

```
jr $ra      #jump back to calling routine.
```

تخصيص مساحة للبيانات الجديدة في stack

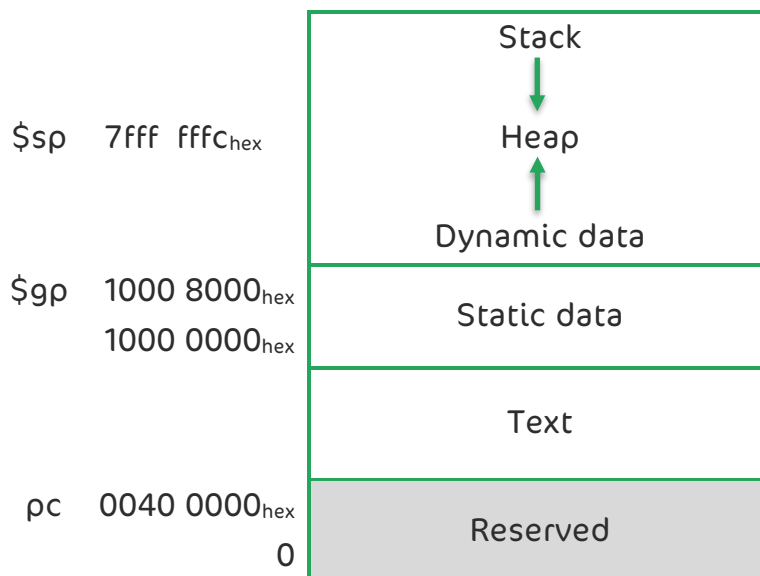
سجل التفعيل (Procedure Frame) هو جزء من المكس (stack) يحتوي على السجلات المحفوظة والمتغيرات المحلية التابع للإجراء procedure.

في MIPS يوجد مؤشر (\$fp) Frame pointer، هذا المؤشر يوفر سجلاً أساسياً ثابت داخل الـ procedure من أجل الذاكرة المحلية: لأن مؤشر stack قد يتغير أثناء التنفيذ Procedure (مثل حالة استدعاء تابع من تابع آخر).



■ ملاحظة أرسيزية:

يستخدم المكس (stack) أيضاً لتخزين المتغيرات المحلية التي لا تتسع في السجلات، مثل المصفوفات المحلية أو الهياكل (structures).



شرح:

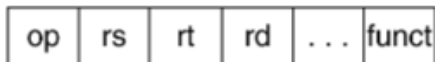
- Reserved: لا نتعامل معها، هي operating system.
- Text: مقطع Unix يحوي على رمز لغة الآلة.
- Static data: تدعى أيضاً (global data) وهي مكان لحفظ المتحولات الثابتة التي يدل عليها (global pointer).
- Dynamic data: مكان لحفظ المتحولات المتغيرة و linked list.
- Heap: هو المكان الذي نحفظ فيه Dynamic data والذي نستخدمه لتوليد object في الذاكرة خلال Runtime.
- و من أجل الاستخدام الأمثل للذاكرة يكون المؤشرين عكس بعضهما لتجنب حدوث overflow.

حفظ: دائماً PC 0x00400000

1. Immediate addressing

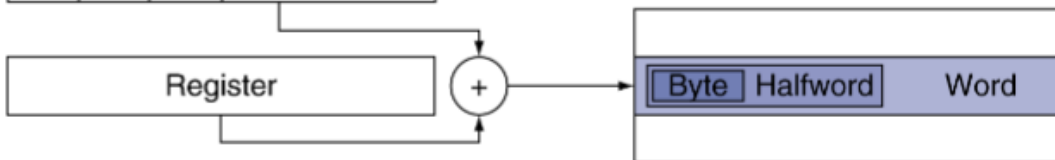
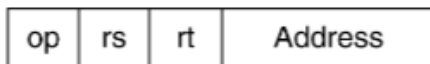


2. Register addressing

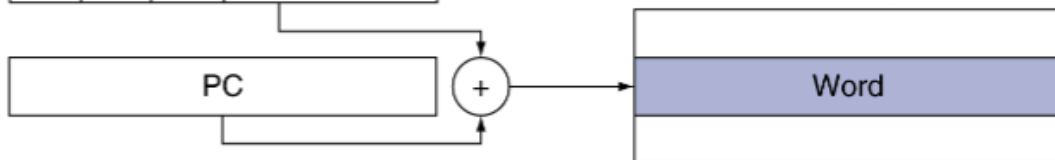
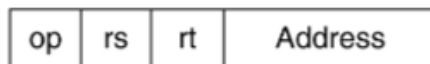


Registers
Register

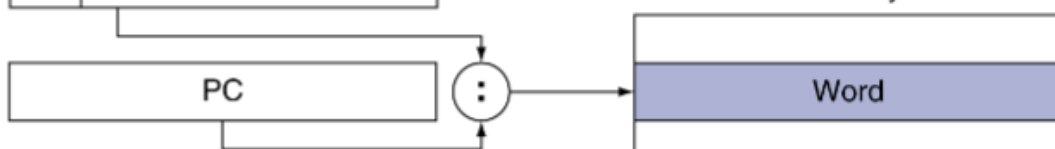
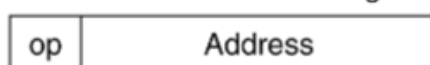
3. Base addressing



4. PC-relative addressing



5. Pseudodirect addressing





■ Addressing Modes:

1. immediate addressing

قيمة فورية وليست عنوان

2. rs rt rd Register addressing

تحتوي على عنوان السجلات وليست قيم

3. Base addressing

إزاحة للعنوان وتستخدم في التعليمات load/store

4. pc-relative addressing.

تستعمل في تعليمات البرانش، العنوان بضربه بـ 4 وجمعه لـ PC

5. pseudo direct addressing.

تستعمل في القفز jump jal وهي خاصة بـ MIPS.

Converting Machine Language into Assembly Language

■ Example:

Here are six machine Language instructions in hex:

فيما يلي ست تعليمات لغة الآلة بالنظام السداسي عشر:

00001025

0005402A

11000003

00441020

20A5FFFF

08100001



Let the first instruction be at address (0x0040000)

لتكن التعليمات الأولى عند هذا العنوان:

■ Next step:

تحويل إلى النظام الثنائي.

Mnemonic ↕	Meaning ↕	Type ↕	Opcode ↕	Funct ↕
add	Add	R	0x00	0x20
addi	Add Immediate	I	0x08	NA
addiu	Add Unsigned Immediate	I	0x09	NA
addu	Add Unsigned	R	0x00	0x21
and	Bitwise AND	R	0x00	0x24
andi	Bitwise AND Immediate	I	0x0C	NA
beq	Branch if Equal	I	0x04	NA
blez	Branch if Less Than or Equal to Zero	I	0x06	NA
bne	Branch if Not Equal	I	0x05	NA
bgtz	Branch on Greater Than Zero	I	0x07	NA
div	Divide	R	0x00	0x1A
divu	Unsigned Divide	R	0x00	0x1B



■ **ملاحظة للمساعدة بالحفظ:**

كل j -type الـ op لها 2, 3.

التعليمات السابقة بالنظام الثنائي

00001000000100000000000000000001

- Next step:
separation of fields:

فصل الحقول بناءً على: opcode

0	0	0	2	0	37
0	0	5	8	0	42
4	8	0	+3		
0	2	4	2	0	32
8	5	5	-1		
2	00000100000000000000000001				

- Next step:
translate to MIPS instructions:
الترجمة لتعليمات MIPS

Assembly Language: Version1

0x00400000	or \$2, \$0, \$0
0x00400004	slt \$8, \$0, \$5
0x00400008	beq \$8, \$0, 3
0x0040000C	add \$2, \$2, \$4
0x00400010	addi \$5, \$5, -1
0x00400014	j 0x100001

Better solution: translate to more meaningful instructions (fix the branch/jump and add labels).
الترجمة لتعليمات أكثر فائدة.

بالسطر الأخير، العنوان 0x100001 يُضرب بـ 4 ليعود لعنوان بداية الـ loop وهو: 0x400004.

Assemble Language: Version2

or \$v0, \$0, \$0

Loop: slt \$t0, \$0, \$a1

beq \$t0, \$0, End_Loop

add \$v0, \$v0, \$a0

addi \$a1, \$a1, -1

j Loop

End_Loop: ...

- Next step:
translate to C code:
Mapping: \$v0: product
\$a0: mcand
\$a1: mplier

```
product=0;
while (mplier > 0){
    product += mcand;
    mplier -= 1;
}
```

Category	Instruction	Example	Meaning
Arithmetic	add	add \$s1,\$s2,\$s3	$s1 = s2 + s3$
	subtract	sub \$s1,\$s2,\$s3	$s1 = s2 - s3$
	add immediate	addi \$s1,\$s2,20	$s1 = s2 + 20$
Data transfer	load word	lw \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	store word	sw \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$
	load half	lh \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	load half unsigned	lhu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	store half	sh \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$
	load byte	lb \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	load byte unsigned	lbu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$
	load linked word	ll \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$
	store condition. word	sc \$s1,20(\$s2)	$\text{Memory}[s2+20]=s1; s1=0 \text{ or } 1$
	load upper immed.	lui \$s1,20	$s1 = 20 * 2^{16}$
Logical	and	and \$s1,\$s2,\$s3	$s1 = s2 \& s3$
	or	or \$s1,\$s2,\$s3	$s1 = s2 s3$
	nor	nor \$s1,\$s2,\$s3	$s1 = \sim (s2 s3)$
	and immediate	andi \$s1,\$s2,20	$s1 = s2 \& 20$
	or immediate	ori \$s1,\$s2,20	$s1 = s2 20$
	shift left logical	sll \$s1,\$s2,10	$s1 = s2 \ll 10$
	shift right logical	srl \$s1,\$s2,10	$s1 = s2 \gg 10$
Conditional branch	branch on equal	beq \$s1,\$s2,25	if ($s1 == s2$) go to PC + 4 + 100
	branch on not equal	bne \$s1,\$s2,25	if ($s1 \neq s2$) go to PC + 4 + 100
	set on less than	slt \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$
	set on less than unsigned	sltu \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$
	set less than immediate	slti \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$
Unconditional jump	jump	j 2500	go to 10000
	jump register	jr \$ra	go to \$ra
	jump and link	jal 2500	$ra = PC + 4$; go to 10000

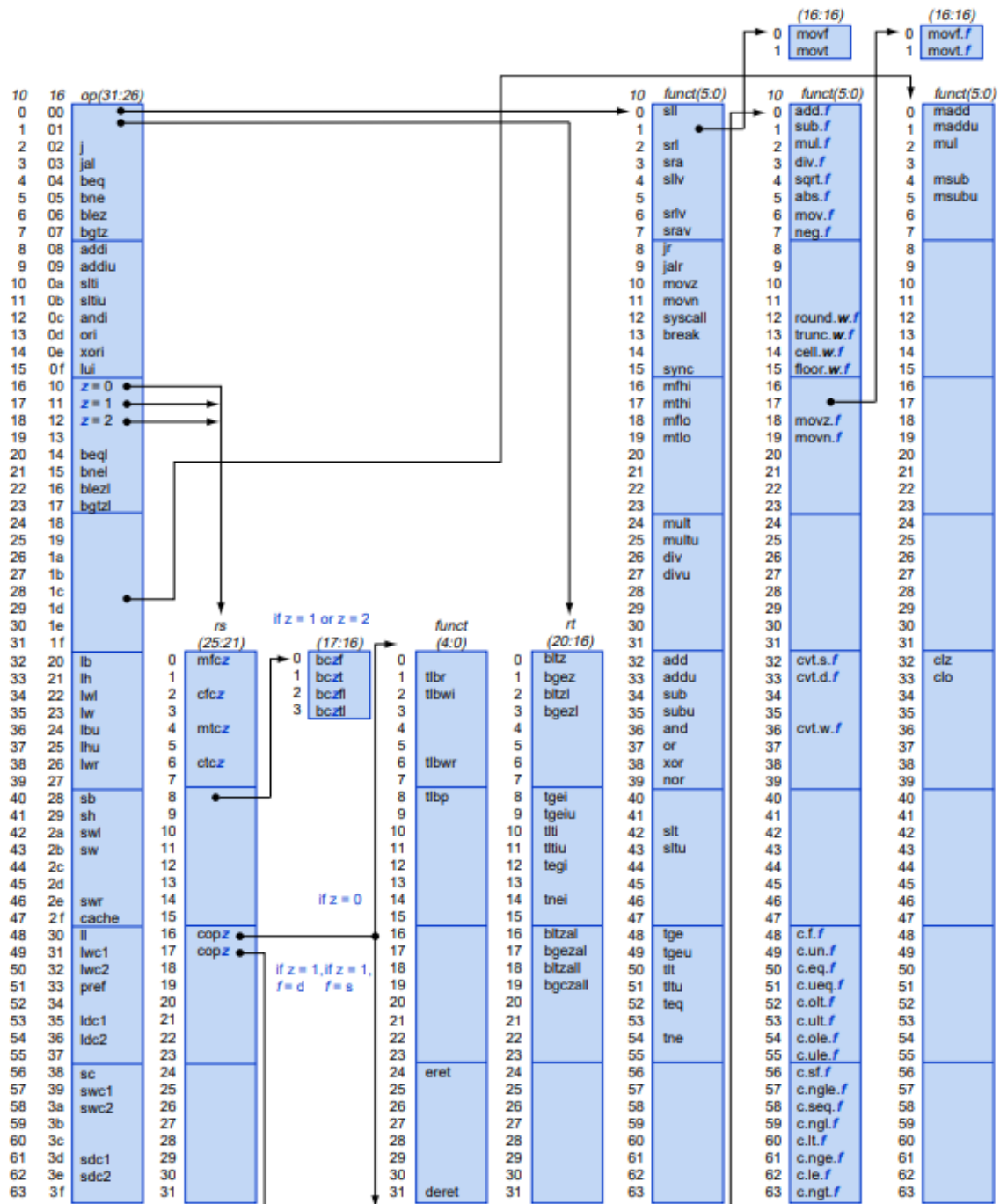


FIGURE A.10.2 MIPS opcode map. The values of each field are shown to its left. The first column shows the values in base 10 and the second shows base 16 for the op field (bits 31 to 26) in the third column. This op field completely specifies the MIPS operation except for 6 op values: 0, 1, 16, 17, 18, and 19. These operations are determined by other fields, identified by pointers. The last field (funct) uses “f” to mean “s” if rs = 16 and op = 17 or “d” if rs = 17 and op = 17. The second field (rs) uses “z” to mean “0”, “1”, “2”, or “3” if op = 16, 17, 18, or 19, respectively. If rs = 16, the operation is specified elsewhere: if z = 0, the operations are specified in the fourth field (bits 4 to 0); if z = 1, then the operations are in the last field with f = s. If rs = 17 and z = 1, then the operations are in the last field with f = d.